

Matière : Système d'exploitation II

TP 3 : : Initiation à la programmation en shell

Enseignante du TP : Rim Chiha

Niveau : LISI1 A, C

Objectifs

- Comprendre les bases de l'écriture d'un script simple.
 - Automatiser une tâche répétitive.
-

1. Qu'est-ce qu'un script Shell ?

Un **script Shell** est un **fichier texte** contenant une suite de **commandes Unix/Linux**, qui s'exécutent automatiquement dans l'ordre. Cela permet d'**automatiser** des tâches répétitives : gestion de fichiers, installation de logiciels, traitement de données.

2. Structure d'un script shell

Un script Shell classique commence toujours par :

```
#!/bin/bash
```

Cette ligne est appelée **shebang**. Elle indique quel interpréteur utiliser (bash, sh, etc.).

Exemple :

```
#!/bin/bash
```

```
echo "Bonjour tout le monde !"
```

3. Rendre un script exécutable

Après avoir écrit ton script (par exemple dans mon_script.sh), tu dois lui donner les droits d'exécution :

```
chmod +x mon_script.sh
```

Pour exécuter le script, vous pouvez utiliser `./mon_script.sh` ou bien `bash mon_script.sh`

4. Variables dans un script

Il existe deux types de variables en script shell :

Variables utilisateurs

Ce sont les variables que tu declares toi-même dans ton script ou dans ton terminal. Elles n'existent que dans le Shell actuel et ne sont pas automatiquement transmises aux autres processus.

Exemples :

- Initialisation de variables

```
nom="Alice"  
age=25
```

- Lire l'entrée utilisateur : demander l'utilisateur de saisir une information à travers l'instruction read

```
read a  
read -p "donner un entier " a
```

- Utilisation de la valeur d'une variable par le symbole `$` : afficher la valeur de la variable a

```
echo "valeur de a est $a "
```

Variables d'environnement

Ce sont des variables systèmes qui affectent le comportement du système ou des processus qui dépendent.

HOME : contient le répertoire personnel de l'utilisateur

PATH : contient la liste des chemins pour chercher les exécutable

USER : contient le nom de l'utilisateur actuellement connecté

SHELL : contient le chemin du programme shell utilisé (/bin/bash)

PWD : contient le répertoire de travail actuel

5. Les opérateurs

Arithmétiques

Addition + Soustraction - Multiplication * Division / Modulo % Puissance **

Comparaison

-eq correspond à =

-ne correspond à !=

-lt correspond à <

-le correspond à <=

-gt correspond à >

-ge correspond à >=

Logique

&& correspond à ET logique

|| correspond à OU logique

! correspond NOT logique

6. Les structures conditionnelles

Une structure conditionnelle en programmation est un mécanisme qui permet d'exécuter différentes instructions selon qu'une condition est vraie ou fausse.

La structure conditionnelle if :

Syntaxe1	Syntaxe2	Syntaxe3
if [condition] ; then instructions fi	if [condition] ; then instructions else instructions fi	if [condition1] ; then instructions elif [condition1] ; then instructions else instructions fi fi

Exercice1

Ecrire un script qui donne la main à l'utilisateur de saisir un entier et affiche un message indiquant si l'entier est paire ou impaire.

```
#!/bin/bash
echo "Veuillez saisir un nombre : "
read nb
if (( nb % 2 == 0 )); then
    echo "Le nombre est pair."
else
    echo "Le nombre est impair."
fi
```

La structure conditionnelle à choix multiples case :

```
case $variable in
    motif1)
        # instructions ;;
    motif2)
        # instructions ;;
    .....
    motifn)
        # instructions ;;
    *)
        # instructions par défaut ;;
esac
```

Exercice 2

Ecrire un script qui demande à l'utilisateur de saisir un numéro du jour et affiche le jour qui correspond à ce numéro. (1→ lundi , 2→mardi, 3→ mercredi, 4→jeudi, 5→vendredi, 6→samedi, 7→ dimanche, autre valeur → erreur)

```
#!/bin/bash
read -p "Entrez un jour de la semaine : " jour
case $jour in
    1)
        echo "C'est lundi!";;
    2)
        echo "C'est mardi" ;;
    3)
        echo "C'est mercredi";;
    4)
        echo "C'est jeudi";;
    5)
        echo "C'est vendredi";;
    6)
        echo "C'est samedi";;
    7)
        echo "C'est dimanche";;
    *)
        echo "Jour non valide. Veuillez entrer un jour de la
semaine." ;;
esac
```

7. Les structures itératives

Une structure répétitive permet de répéter automatiquement un ensemble d'instructions plusieurs fois tant qu'une condition est remplie.

La boucle while

```
while [ condition ]; do
    # instructions répétées
done
```

Exercice 3

Ecrire un script qui permet d'afficher un entier à l'envers (utiliser la boucle while)

Exemple N=123456 resultat 654321

```
#!/bin/bash
echo "Veuillez saisir un entier : "
read nb
# Initialiser une variable pour stocker le nombre inversé
```

```

inverse=""
# Boucle pour inverser le nombre
while [ $nb -ne 0 ];
do
    U=$(( $nb % 10 ))
    inverse="$inverse$U"
    nb=$(( $nb / 10 ))
done
echo "Le nombre à l'envers est : $inverse"

```

La boucle for

Syntaxe1	Syntaxe2	Syntaxe3
<pre> for variable in liste_de_valeurs; do #instructions done </pre>	<pre> for ((initialisation; condition; incrémentation)); do # instructions done </pre>	<pre> for variable in {debut..fin}; do # instructions done </pre>

Exercice 4

Ecrire un script qui permet de calculer la factorielle d'un entier positif saisi par l'utilisateur. Si l'utilisateur introduit un entier négatif alors on affiche un message d'erreur et on arrête l'exécution.

```

#!/bin/bash
echo "Veuillez saisir un entier positif : "
read nb
# Vérifier si l'entrée est un nombre entier positif
if [ "$nb" -lt 0 ]; then
    echo "Erreur : Vous devez saisir un entier positif."
    exit 1
fi
fact=1

for (( i=1; i<=nb; i++ ))
do
    fact=$((fact * i))
done
echo "La factorielle de $nombre est : $fact"

```

8. Les chaînes de caractères

Les chaînes de caractères en Shell sont des séquences de caractères alphanumériques et de symboles. Elles peuvent être utilisées pour manipuler du texte, créer des variables, ou interagir

avec des fichiers et des entrées-sorties. En Shell, on peut comparer des chaînes à l'aide de tests conditionnels :

- Égalité : = ou ==
- Différence : !=

Les fonctions prédéfinies :

1. Accéder à une sous chaîne : `${chaîne:position_de_départ:longueur}`

Position_de_départ : L'indice à partir duquel commencer à extraire (les indices commencent à 0). Longueur : Le nombre de caractères à extraire.

2. Longueur d'une chaîne : `${#chaîne}`

3. Recherche d'une sous chaîne dans une **chaîne** :

```
[[ "$chaîne" =~ "sous_chaîne" ]]
```

4. Remplacer une chaîne par une autre chaîne

`${chaîne/ancienne/nouvelle}` : remplacer une seule fois

`${chaîne//ancienne/nouvelle}` : remplacer toutes les occurrences

5. Supprimer une sous chaîne

`${chaîne/sous chaîne/}` : supprimer une seule fois

`${chaîne//sous chaîne/}` : supprimer toutes les occurrences

6. Extraire une sous chaîne à partir d'une position jusqu'à la fin de la chaîne

`${chaîne:position_de_départ}`

7. convertir en majuscule

`${chaîne^^}`

8. convertir en minuscule

`${chaîne,,}`

9. Concatener deux chaînes : juxtaposer les variables contenant les chaînes

```
#!/bin/bash
chaîne1="Bonjour"
chaîne2="le monde"
# Concaténation des deux chaînes
chaîne3="$chaîne1 $chaîne2"
echo "$chaîne3"
```

Exercice 5

Soit la chaîne de caractères Ch = " je ne suis pas un noir je ne suis pas un rouge je ne suis pas un blanc "

- 1) Compter le nombre d'occurrence du motif "je ne suis pas "

- 2) Remplacer chaque occurrence de ce motif par un espace
- 3) Supprimer les espaces
- 4) Afficher la longueur du chaîne résultat
- 5) Convertir la chaîne résultat en majuscule

```
#!/bin/bash
chaîne="je ne suis pas un noir je ne suis pas un blanc je ne
suis pas un rouge"
# Compter le nombre d'occurrences du motif "je ne suis pas"
cmp=0
pattern="je ne suis pas"
# Boucle pour compter et supprimer les occurrences
while [[ "$chaîne" =~ $pattern ]]; do
    ((cmp++))
    chaîne="${chaîne/$pattern/ }"
done
echo "Le motif 'je ne suis pas' apparaît $cmp fois."
#supprimer les espaces
chaîne_sans_espaces="${chaîne//[[:space:]]/}"
echo "La chaîne après suppression des espaces
$chaîne_sans_espaces "
echo "La longueur du chaîne résultat ${#chaîne_sans_espaces}"
# Convertir la chaîne résultat en majuscule
chaîne_sans_espaces=${chaîne_sans_espaces^^}
echo "La chaîne majuscule : $chaîne_sans_espaces"
```

9. Les tableaux

En script shell (Bash), les *tableaux* sont des structures simples pour stocker plusieurs éléments.

Déclaration d'un tableau

```
mon_tableau=(element1 element2 element3)
tab=() # tab est un tableau vide
tab=(1 2 3) # tab est un tableau rempli par des entiers
tab=(lundi mardi mercredi) # tab est un tableau rempli par des
chaînes de caractères
```

Remplir un tableau par N élément

```
for (( i=0 ; i<N ; i++))
read -p "donner un élément " tab[i]
```

Accéder à un élément dans un tableau

```
Nom_tableau[indice] #indice commence par 0
Tab[i]
```

Afficher tous les éléments du tableau

```
echo "${tab[@]}"
```

Taille du tableau

```
${#tab[@]}
```

Ajouter des éléments

```
tab+=("element4")
```

Parcours du tableau

```
for elem in "${tab[@]}; do
    echo $elem
done
```

Supprimer un élément

```
unset tab[indice]
unset tab[1] # Supprime 'element2'
```

Exercice 6

Ecrire un script qui permet de remplir un tableau par N entiers puis faire la permutation circulaire du tableau.

```
#!/bin/bash
read -p "donner le nombre d'éléments dans le tableau" n
T=()
for ((i=0;i<n; i++))
do
read -p "donner element d'indice $i " T[$i]
done
echo "${T[@]}"
last_indice=$((n-1))
last_element=${T[$last_indice]}
for ((i=$last_indice; i>0;i--))
do
T[$i]=${T[$((i-1))]}

```

```
done
T[0]=$last_element
echo "${T[@]}"
```